

Contemporary C++: *Learning Modern C++ in a Modern Way*



مدرس: سعید امراللہی بیوکی

Saeed Amrollahi Boyouki

www.Linkedin.com/in/saeed-amrollahi-boyouki

Agenda 17/30

Session 17. Introduction to Preprocessors, Separate Compilations, and Components

- The C preprocessor and header files
- Separate compilation: one-definition rule, header files, and source files
- Separate compilation: The stack example
- References and further readings
- Q&A

120 min (incl. Q & A)



Preprocessing and Preprocessor

- Preprocessor: the part of a C++ implementation that *removes comments*, performs *macro substitution*, and *file inclusion*.

C Preprocessor

- C Provides certain language facilities by means of a *preprocessor*, which is conceptually a separate first step in compilation.
- Actually C preprocessor is a system software which is usually called *Macro processor*. Briefly It is called *Cpp*.

- Cpp is character and file oriented.
- Cpp has three fundamental features:
 - File inclusion
 - Macro substitution: Expand Inline
 - Conditional Inclusion

```
#ifndef HEADER_FILE
#define HEADER_FILE

#endif // HEADER_FILE
```

```
#ifndef HEADER_FILE
#define HEADER_FILE

#endif // HEADER_FILE
```

```
#include <stdio.h>
main()
{
    printf("hello, world\n");
}
```

```
#include "filename"
```

```
#include <filename>
```

```
#define name replacement text
```

```
#define PI 3.14159
```

```
#define MAX(A, B) ((A) > (B) ? (A) : (B))
```

C++ Program

- **One Definition Rule (ODR)**: there must be exactly one definition of each entity in a program. If more than one definition appears, say because of replication through header files, the meaning of all such duplicates must be identical.

```
// file1.c:  
struct S1 { int a; char b; };  
struct S1 { int a; char b; }; // error: double definition
```

```
// file1.c:  
struct S2 { int a; char b; };  
// file2.c:  
struct S2 { int a; charb bb; }; // error
```

- A (C/C++) program is a collection of separately compiled units combined by linker. Every function, object, type, etc. used in this collection must have a unique definition. The program must contain exactly one function called main()

Physical structure of C++ programs

Physical structure of C++ programs

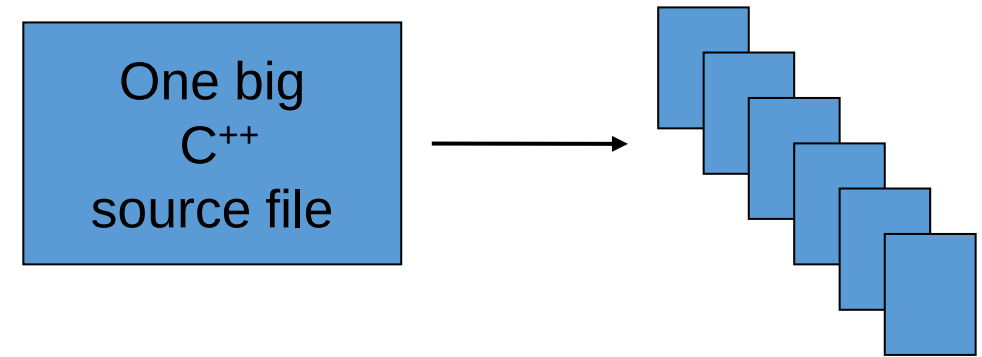
- A file is the traditional unit of storage (in a file system) and the traditional unit of compilation.

Physical structure of C++ programs

- A file is the traditional unit of storage (in a file system) and the traditional unit of compilation.
- Having a complete program in one file is usually impossible.
 - Standard library code
 - Operating system code

Physical structure of C++ programs

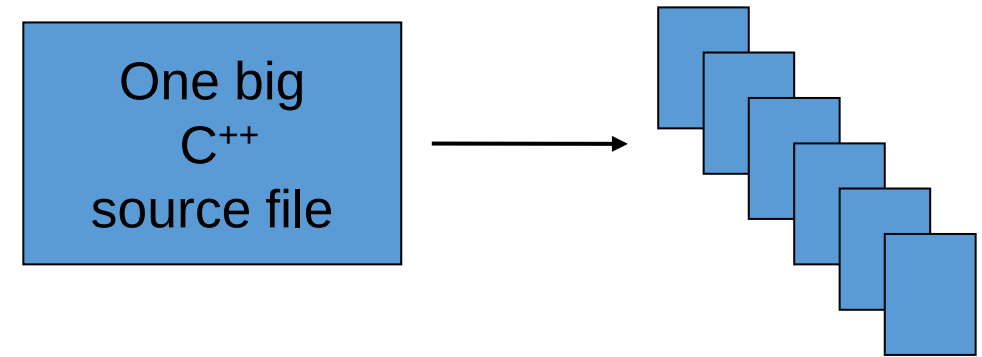
- A file is the traditional unit of storage (in a file system) and the traditional unit of compilation.
- Having a complete program in one file is usually impossible.
 - Standard library code
 - Operating system code



several smaller C++ source files

Physical structure of C++ programs

- A file is the traditional unit of storage (in a file system) and the traditional unit of compilation.
- Having a complete program in one file is usually impossible.
 - Standard library code
 - Operating system code
- Reduced compilation time
- More readability

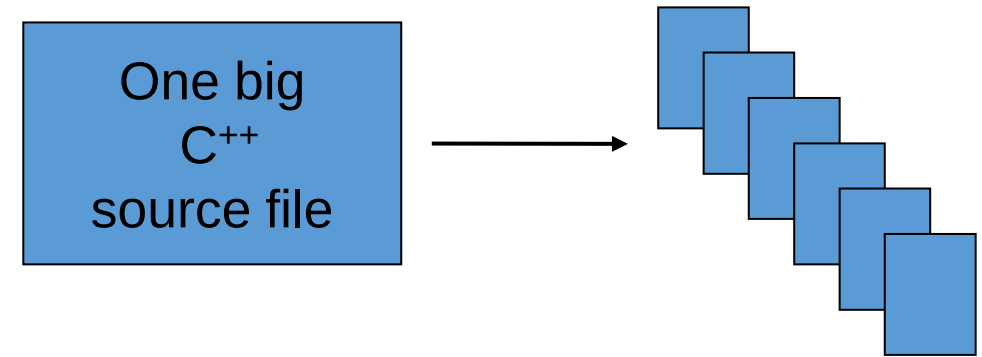


several smaller C++ source files

Physical structure of C++ programs

- A file is the traditional unit of storage (in a file system) and the traditional unit of compilation.
- Having a complete program in one file is usually impossible.

- Standard library code
- Operating system code



several smaller C++ source files

- Reduced compilation time
- More readability

Logical structure $\leftrightarrow$ Physical structure

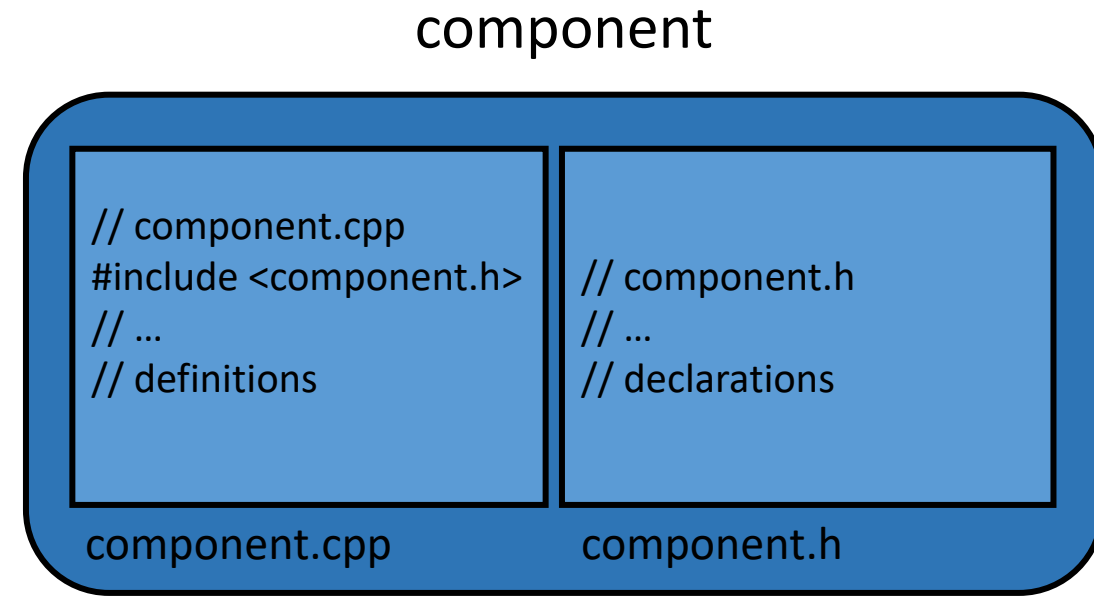
Ccomponents

Components

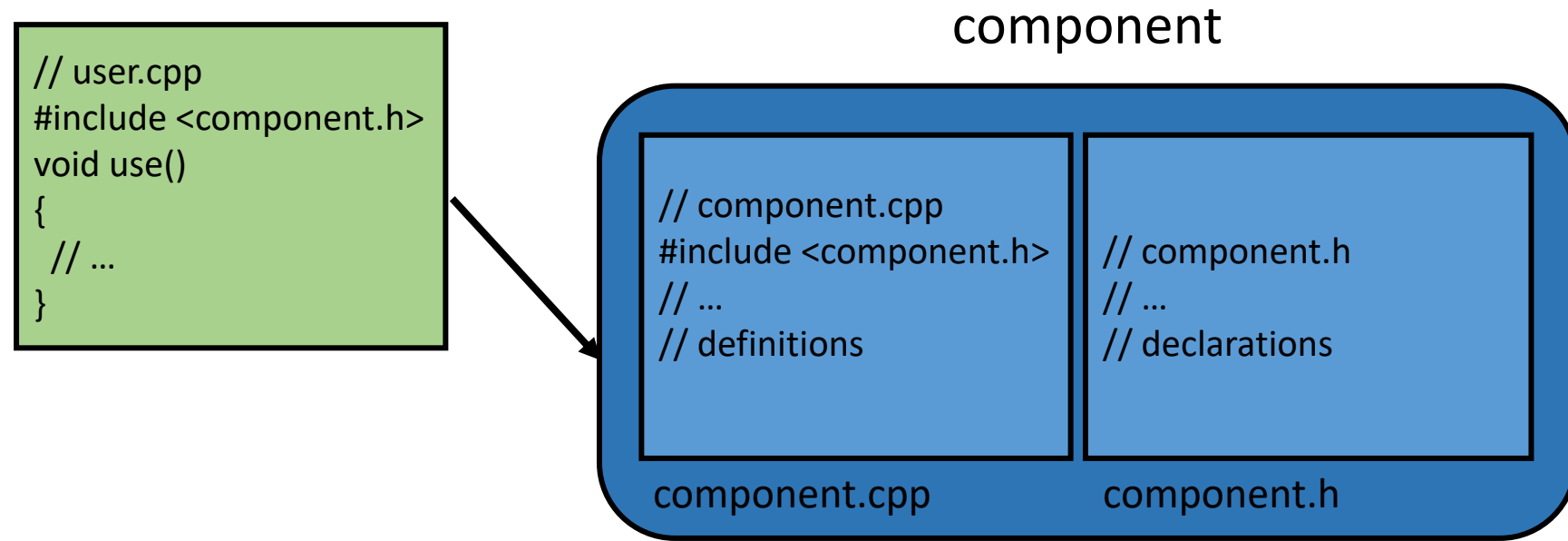
component



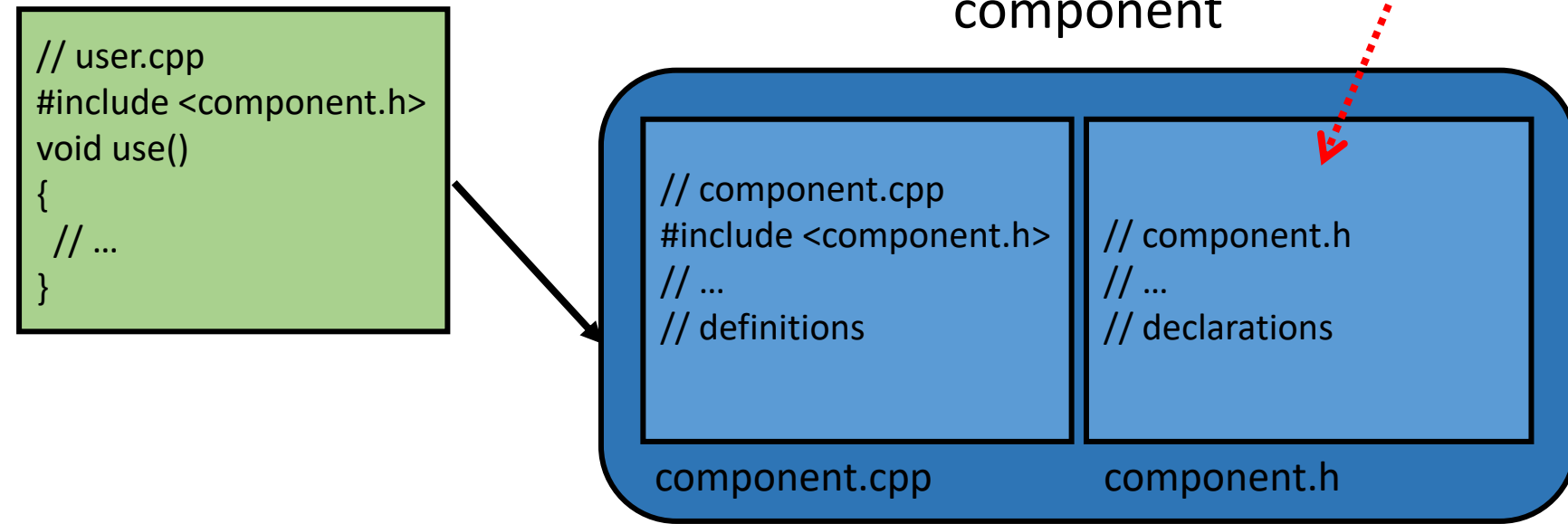
Ccomponents



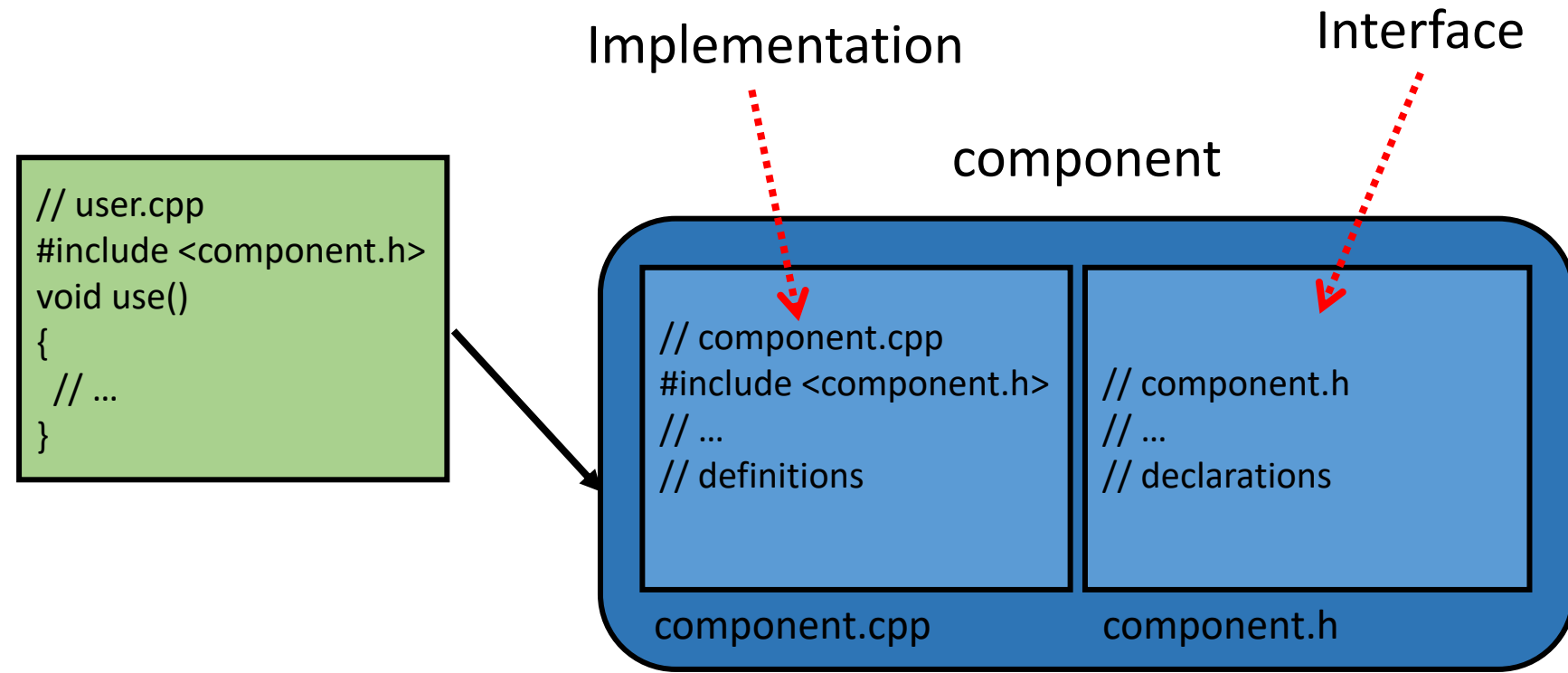
Ccomponents



C Components



C Components



C Components

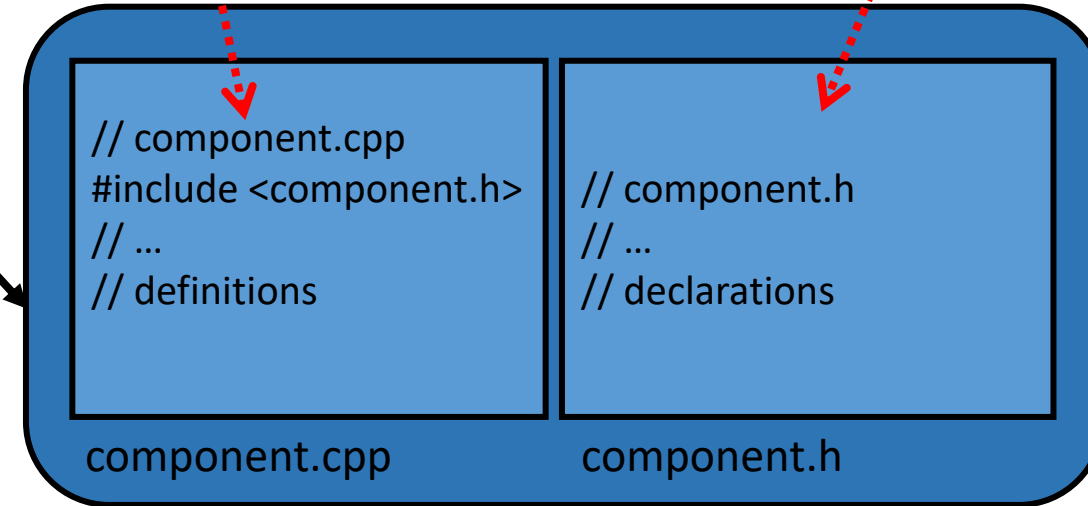
Test driver

```
// user.cpp
#include <component.h>
void use()
{
    // ...
}
```

Implementation

Interface

component



C Components

Test driver

```
// user.cpp
#include <component.h>
void use()
{
    // ...
}
```

Implementation

Interface

component

```
// component.cpp
#include <component.h>
// ...
// definitions
```

component.cpp

```
// component.h
// ...
// declarations
```

component.h

- The Fundamental unit of design

C Components

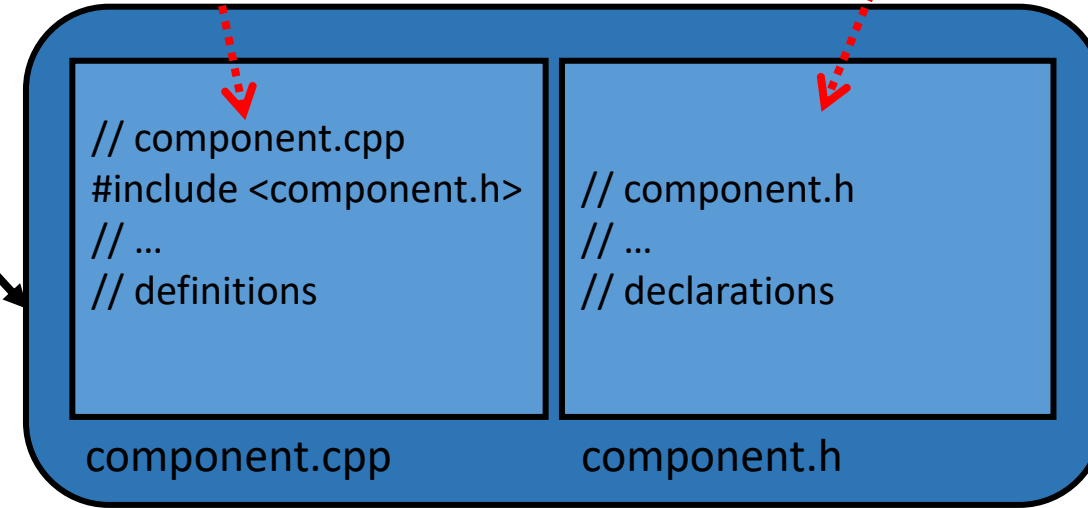
Test driver

```
// user.cpp
#include <component.h>
void use()
{
    // ...
}
```

Implementation

Interface

component



- The Fundamental unit of design
- Physical component vs. Logical component
- Logical vs. physical aspects

C Components

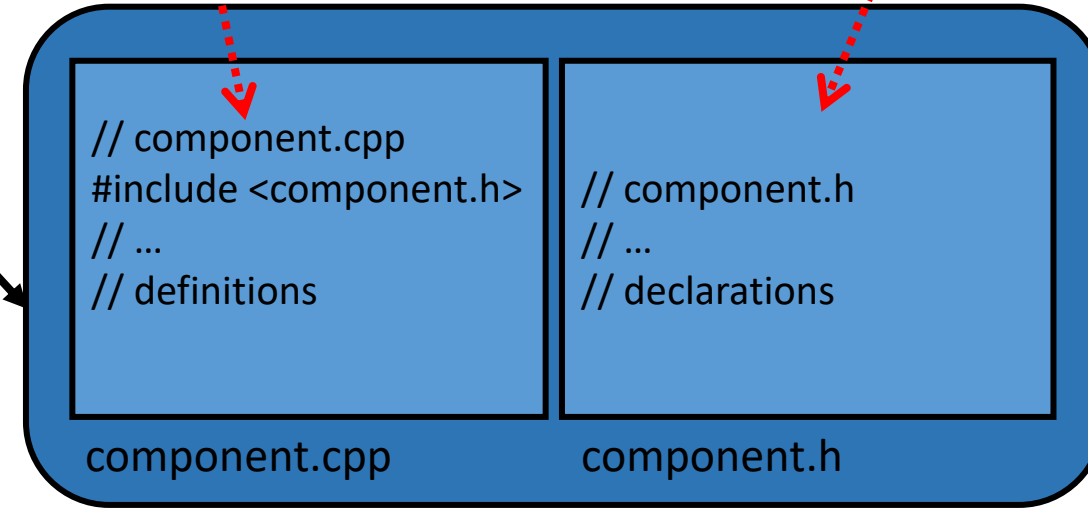
Test driver

```
// user.cpp
#include <component.h>
void use()
{
    // ...
}
```

Implementation

Interface

component



- The Fundamental unit of design
- Physical component vs. Logical component
- Logical vs. physical aspects
- Large-scale C++ software development

C Components

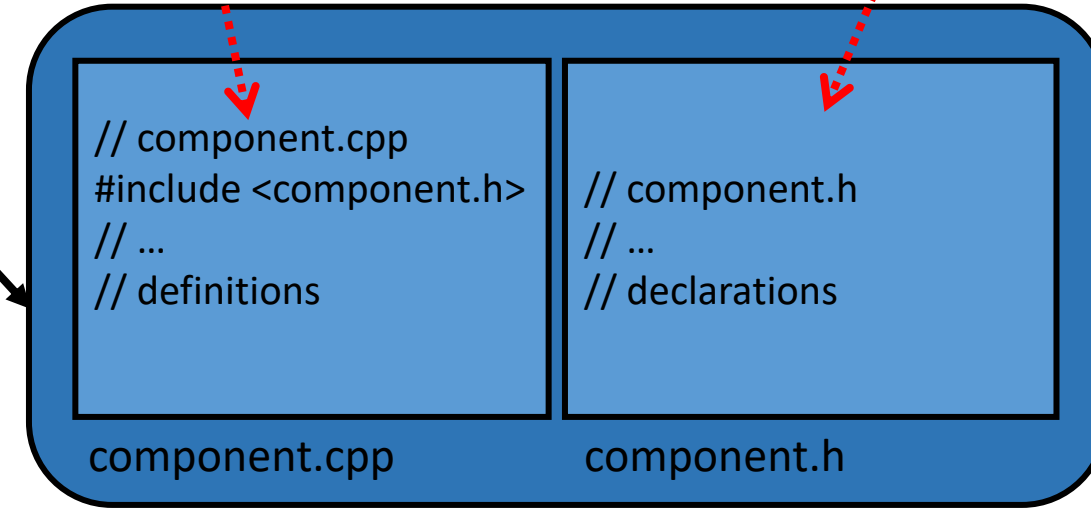
Test driver

```
// user.cpp
#include <component.h>
void use()
{
    // ...
}
```

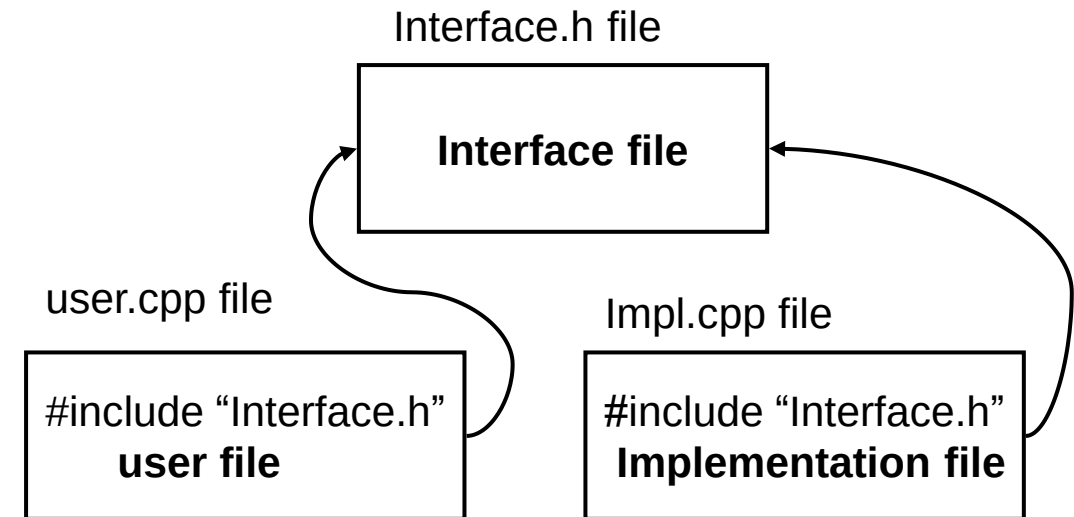
Implementation

Interface

component



- The Fundamental unit of design
- Physical component vs. Logical component
- Logical vs. physical aspects
- Large-scale C++ software development



R references and further readings

[Str13] Bjarne Stroustrup. The C++ Programming Language. 4th edition. Addison Wesley. Reading Mass. USA. May 2013.

[Str22] Bjarne Stroustrup. A Tour of C++, 3rd edition, Addison-Wesley, September 2022.

Talk at CppCon: The C++ Conference.

John Lakos. Advanced Levelization Techniques- Part 1 (Video), September 2016.

Thanks for your patience ...

A man who asks a question is a fool for minute,
The man who does not ask, is a fool for a life.
- Confucius

Learning to ask the right (often hard) questions is an essential part of learning to think as a programmer.

- Bjarne Stroustrup *programming Principles and Practice Using C++, page 4.*

There is no stupid question, but there is stupid answer.
- Howard Hinnant

